

CHPC Guide

Running the Life Engine on the CHPC

This guide walks you through setting up Node.js on the CHPC (Centre for High Performance Computing) and running the Life Engine simulation in headless mode — with no browser or display required.

Table of Contents

1. [Prerequisites](#)
 2. [Installing NVM \(Node Version Manager\)](#)
 3. [Installing Node 16 with NVM](#)
 4. [Cloning / Copying the Project](#)
 5. [Installing Project Dependencies](#)
 6. [Running the Simulation in Headless Mode](#)
 7. [Headless Mode Flags Reference](#)
 8. [Submitting a Job with PBS](#)
 9. [Customising the PBS Script](#)
 10. [Understanding the Output](#)
 11. [Troubleshooting](#)
-

1. Prerequisites

- An active CHPC account with SSH access
- Access to the `CSCI1142` project allocation
- Basic familiarity with the terminal / bash

All commands in this guide are run on the **CHPC login node** unless otherwise stated.

2. Installing NVM (Node Version Manager)

NVM lets you install and switch between Node.js versions without needing `sudo` or system-wide changes. Install it into your home directory as follows.

Step 1 — Download and run the NVM install script

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash
```

The script will add the following lines to your shell profile (`~/.bashrc` , `~/.bash_profile` , or `~/.zshrc`). If it does not, add them manually:

```
export NVM_DIR="$HOME/.nvm"  
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"  
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
```

Step 2 — Reload your shell

```
source ~/.bashrc
```

Step 3 — Verify NVM is installed

```
nvm --version
```

Expected output (version number may vary):

```
0.39.7
```

Note: If `nvm: command not found` , log out and back in, or manually run `source ~/.bashrc` .

3. Installing Node 16 with NVM

The simulation requires **Node.js v16**. Install it with:

```
nvm install 16
```

NVM will download and compile Node 16, then set it as the active version. Verify the installation:

```
nvm use 16
```

```
node --version
```

Expected output:

```
v16.20.2
```

Also confirm the path to the node binary (you will need this for the PBS script):

```
which node
```

Expected output:

```
/home/<your-username>/.nvm/versions/node/v16.20.2/bin/node
```

Making Node 16 the default

So that Node 16 is automatically active in every new shell session:

```
nvm alias default 16
```

4. Cloning / Copying the Project

If the repository is on GitHub:

```
git clone <repository-url> ~/LifeEngine  
cd ~/LifeEngine
```

If you are transferring files from your local machine, use `scp` or `rsync` :

```
# Run this on your LOCAL machine, not the CHPC  
scp -r /path/to/LifeEngine <your-username>@login.chpc.ac.za:~/LifeEngine
```

5. Installing Project Dependencies

The simulation uses webpack and jQuery. Install all dependencies with npm:

```
cd ~/LifeEngine
npm install
```

Do not run `npm run build` — the headless runner (`src/headless.js`) requires the raw source files directly, not the webpack bundle. The build step is only needed for the browser version.

6. Running the Simulation in Headless Mode

The headless runner (`src/headless.js`) runs the simulation as a pure Node.js script with no browser, no canvas, and no display required. It reads CLI flags, simulates for the requested number of ticks, then writes results to a JSON file.

Basic command

```
node src/headless.js --max-ticks 100000
```

With all options

```
node src/headless.js \
  --max-ticks 1000000 \
  --output results.json \
  --config Experiment1_Base.json \
  --load saved_env.json \
  --width 500 \
  --height 500 \
  --cell-size 4 \
  --log-every 10000
```

Example: run for 500k ticks on a 500×500 grid

```
node src/headless.js --max-ticks 500000 --width 500 --height 500 --output
experiment_run1.json
```

Example: run on a wide rectangular grid (1000 columns, 300 rows)

```
node src/headless.js --max-ticks 500000 --width 1000 --height 300 --output
wide_run.json
```

Example: load a saved environment and continue for 200k more ticks

```
node src/headless.js --load Experiment1_Base.json --max-ticks 200000 --  
output continued.json
```

7. Headless Mode Flags Reference

Flag	Required	Default	Description
<code>--max-ticks <N></code>	Yes	—	Stop after this many simulation ticks. The simulation also stops early if all organisms go extinct.
<code>--output <file></code>	No	<code>results.json</code>	Path (relative to project root) where the JSON results file is written.
<code>--config <file></code>	No	<i>(defaults)</i>	Path to a JSON file containing hyperparameter overrides. Accepts either a plain hyperparameter object or a full serialised environment file (the hyperparams are read from its <code>controls</code> field).
<code>--load <file></code>	No	<i>(fresh start)</i>	Path to a serialised environment JSON file to resume from. If omitted, the simulation starts from scratch with a single organism at the origin.
<code>--width <N></code>	No	<i>(renderer default)</i>	Grid width in columns. Can be set independently of <code>--height</code> . Ignored if <code>--load</code> is used.
<code>--height <N></code>	No	<i>(renderer default)</i>	Grid height in rows. Can be set independently of <code>--width</code> . Ignored if <code>--load</code> is used.
<code>--cell-size <N></code>	No	4	Pixel size of each grid cell. A 500×500 grid at cell size 4 covers a 2000×2000 virtual canvas.
<code>--log-every <N></code>	No	10000	Print a progress line to stdout every N ticks. Set to 0 to silence progress output.

What the progress log looks like

```
[headless] tick=10000/1000000 pop=2122 species=127 1940 ticks/s  
elapsed=5.2s
```

Field	Meaning
tick	Current tick / maximum ticks
pop	Number of live organisms
species	Number of extant species
ticks/s	Simulation throughput
elapsed	Wall-clock time since start

Early stopping

If every organism dies before `--max-ticks` is reached, the simulation stops immediately and reports the extinction tick:

```
[headless] Extinction at tick 43217 – stopping early.
```

8. Submitting a Job with PBS

The file `run_simulation.pbs` in the project root is a ready-to-use PBS job script. It handles:

- Setting the project allocation (CSCI1142)
- Requesting compute resources
- Locating the correct Node binary
- Running the headless simulation
- Email notifications on job start, end, and abort

Step 1 — Edit the script

Open `run_simulation.pbs` in a text editor and update the variables at the top of the file:

```
nano ~/LifeEngine/run_simulation.pbs
```

The key variables to change:

```
MAX_TICKS=1000000          # how many ticks to run
OUTPUT="results_${PBS_JOBID}.json" # output file name (PBS_JOBID is auto-
filled)
CONFIG=""                   # path to hyperparameter JSON, or leave empty for
defaults
LOAD=""                     # path to a saved environment JSON, or leave
empty for fresh start
LOG_EVERY=10000             # progress log interval

NODE_BIN="/home/<your-username>/.nvm/versions/node/v16.20.2/bin/node"
PROJECT_DIR="/home/<your-username>/LifeEngine"
```

Important: Replace `<your-username>` with your actual CHPC username in both `NODE_BIN` and `PROJECT_DIR`.

Also update the email address for notifications:

```
#PBS -M your-email@students.uct.ac.za
```

Step 2 — Create the logs directory

The PBS script writes stdout and stderr logs into a `logs/` subdirectory. Create it before submitting:

```
mkdir -p ~/LifeEngine/logs
```

Step 3 — Submit the job

```
cd ~/LifeEngine
qsub run_simulation.pbs
```

You will receive a job ID, e.g.:

```
1234567.sched1.chpc.ac.za
```

Step 4 — Monitor the job

Check if your job is queued or running:

```
qstat -u <your-username>
```

Stream the live log output:

```
tail -f ~/LifeEngine/logs/life_engine_<JOBID>.out
```

Check for errors:

```
cat ~/LifeEngine/logs/life_engine_<JOBID>.err
```

Step 5 — Cancel a job (if needed)

```
qdel <JOBID>
```

9. Customising the PBS Script

The PBS directives at the top of `run_simulation.pbs` control how the job is scheduled. Here is what each line means and how to adjust it.

```
#PBS -N life_engine          # Job name shown in qstat
#PBS -P CSCI1142             # Project allocation – do not change
#PBS -l walltime=24:00:00    # Maximum wall time (HH:MM:SS). Increase for
longer runs.
#PBS -l nodes=1:ppn=1        # 1 node, 1 CPU core (simulation is single-
threaded)
#PBS -l mem=4gb              # Memory allocation. 4 GB is sufficient for most
runs.
#PBS -q serial               # Queue to use. 'serial' is for single-core jobs.
#PBS -o logs/...             # Stdout log file path
#PBS -e logs/...             # Stderr log file path
#PBS -m abe                  # Email on: (a)bort, (b)egin, (e)nd
#PBS -M your@email.com       # Email address for notifications
```

Estimating walltime

Based on local testing, the simulation runs at roughly **1,000–2,000 ticks per second** (varies with population size). Use the table below as a guide:

Ticks	Estimated time
100,000	~1 minute

Ticks	Estimated time
1,000,000	~10–15 minutes
5,000,000	~1–2 hours
20,000,000	~4–8 hours

Set `walltime` with a comfortable margin above your estimate. Jobs that exceed their walltime are killed automatically.

Running multiple experiments

To run several parameter conditions in parallel, make a copy of the PBS script for each experiment:

```
cp run_simulation.pbs run_exp2.pbs
nano run_exp2.pbs    # change MAX_TICKS, CONFIG, OUTPUT
qsub run_exp2.pbs
```

10. Understanding the Output

When the simulation finishes, a JSON results file is written (e.g. `results_1234567.json`). The top-level structure is:

```
{
  "total_ticks": 1000000,
  "elapsed_seconds": 612.4,
  "ticks_per_second": 1633,
  "extinction_tick": null,
  "reached_max_ticks": true,
  "final_population": 2417,
  "final_species": 203,
  "fossil_record": { ... }
}
```

Field	Description
<code>total_ticks</code>	Number of ticks actually completed
<code>elapsed_seconds</code>	Wall-clock time the simulation ran for
<code>ticks_per_second</code>	Average simulation throughput

Field	Description
extinction_tick	Tick at which all organisms died, or <code>null</code> if they survived to the end
reached_max_ticks	<code>true</code> if the run completed normally, <code>false</code> if extinction occurred first
final_population	Number of organisms alive at the end
final_species	Number of extant species at the end
fossil_record	Full time-series data (populations, species counts, diet breakdowns, mutation rates)

The `fossil_record` object

The fossil record contains parallel arrays recorded every 100 ticks (configurable via `data_update_rate` in the source):

```
"fossil_record": {
  "tick_record":          [0, 100, 200, ...],
  "pop_counts":           [1, 3, 12, ...],
  "species_counts":       [1, 1, 2, ...],
  "av_mut_rates":         [5.0, 5.2, ...],
  "av_cells":             [3.0, 3.1, ...],
  "species_diet_counts":  [{ "type0_only": 1, "generalist": 0, ... },
  ...],
  "population_diet_counts": [{ "type0_only": 1, "generalist": 0, ... },
  ...]
}
```

You can load and analyse this in Python:

```
import json
import matplotlib.pyplot as plt

with open("results_1234567.json") as f:
    data = json.load(f)

record = data["fossil_record"]
plt.plot(record["tick_record"], record["pop_counts"])
plt.xlabel("Tick")
plt.ylabel("Population")
plt.title("Population over time")
plt.savefig("population.png")
```

11. Troubleshooting

nvm: command not found after installing

Run `source ~/.bashrc` (or log out and back in). If the problem persists, check that the NVM lines were added to `~/.bashrc`:

```
grep -n "NVM" ~/.bashrc
```

If there are no results, add them manually:

```
echo 'export NVM_DIR="$HOME/.nvm"' >> ~/.bashrc
echo '[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"' >> ~/.bashrc
source ~/.bashrc
```

ERROR: Node binary not found at ...

The `NODE_BIN` path in the PBS script does not match where NVM installed Node. Find the correct path on the CHPC and update the script:

```
nvm use 16
which node
```

Copy the output into the `NODE_BIN` variable in `run_simulation.pbs`.

Cannot find module './WorldConfig' or similar

The job is not running from the project root. Make sure `PROJECT_DIR` in the PBS script is set to the **absolute path** of your project directory (e.g. `/home/tmackay/LifeEngine`), and that `cd "$PROJECT_DIR"` succeeds.

Job is killed before finishing (walltime exceeded)

Increase the walltime in the PBS script:

```
#PBS -l walltime=48:00:00
```

Or reduce `MAX_TICKS` so the job finishes in time.

Output JSON is missing or empty

Check the error log for the job:

```
cat ~/LifeEngine/logs/life_engine_<JOBID>.err
```

Common causes:

- The `logs/` directory does not exist — run `mkdir -p ~/LifeEngine/logs`
 - The output path has a permission issue — use a relative path inside the project directory
 - The simulation crashed — look for a JavaScript error or stack trace in the `.err` file
-

Checking your allocation

To confirm your project is active and check remaining compute hours:

```
accounts
```